

Daily Progress Report

Week 5_Day2: State Management & Event Handling

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_5 (Day 2)
Date	22 July, 2026

Day 2: State Management & Event Handling

1.1 Topics Studied

- Introduction to the **useState** hook for managing local component state
- Handling user events in React (such as **onClick** and **onChange**)
- Rendering lists and dynamically mapping over arrays using **.map()**
- Understanding the importance of unique **key** props in list rendering
- Conditional rendering in JSX to show or hide UI elements based on state
- **Single Source of Truth** concept in state management
- Navigating terminal directories and resolving common CLI path errors (**ENOENT**) in Vite

1.2 Task Description

Developed an interactive **FAQ Accordion** component to demonstrate local state management and event handling in React. The component mapped over an array of question and answer objects to dynamically generate the UI. Local state was implemented to track which accordion item was currently expanded, with specific logic ensuring that only one item could be open at a time, automatically collapsing any previously opened item.

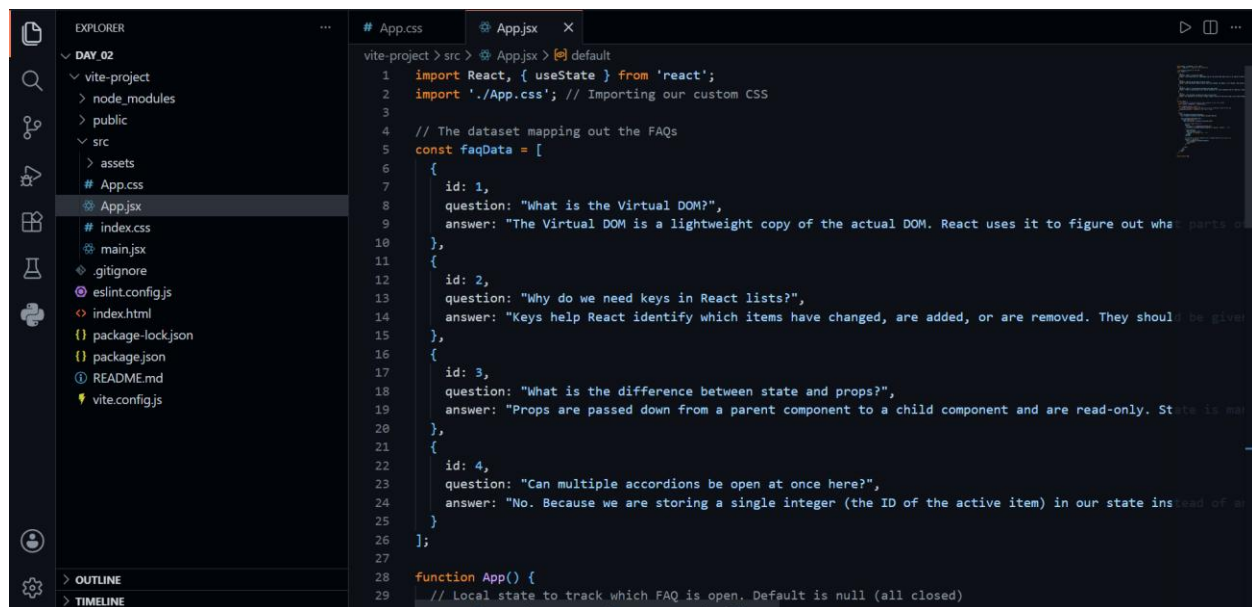
1.3 Implementation Details

- Configured a new React component environment using Vite and resolved directory traversal issues to successfully start the local development server.
- Defined a structured data array containing multiple FAQ objects, each with a unique id, question, and answer.
- Utilized the `.map()` method to iterate over the data array and dynamically render the accordion interface, assigning stable key props to each mapped element.
- Implemented the `useState` hook to initialize a single `openId` state variable, defaulting to null to ensure all accordion items remain closed on the initial render.
- Created a `toggleAccordion` event handler function that uses the previous state updater (`prevId`) to safely determine whether to open a new accordion item or close the currently active one.
- Attached the event handler to the accordion trigger buttons using an inline arrow function on the `onClick` attribute to pass specific IDs without invoking the function during render.
- Applied conditional rendering logic (`openId === faq.id &&`) to display the answer content only for the currently active accordion item.
- Integrated custom CSS to provide a clean, modern layout with hover states, active tab indicators (switching between '+' and '-'), and a smooth slide-down animation for the content.

1.4 Outcome

Successfully developed a fully functional, interactive **FAQ Accordion** component that practically demonstrates core React state management principles. The project solidified the understanding of how to capture user interactions through event handlers and use them to trigger component re-renders via the **useState** hook. By enforcing a single active ID in state, the task reinforced the concept of maintaining a minimal **Single Source of Truth**. Additionally, navigating terminal path errors provided practical experience in managing a modern **Vite**-based **Node.js** project environment.

Screenshots:



The screenshot shows a VS Code editor with a project named 'vite-project'. The Explorer sidebar on the left shows the file structure: DAY_02, vite-project, node_modules, public, src, assets, App.css, App.jsx, index.css, main.jsx, .gitignore, eslint.config.js, index.html, package-lock.json, package.json, README.md, and vite.config.js. The main editor area shows the code for App.jsx. The code imports React and useState from 'react', imports './App.css', and defines a dataset of FAQs. The App() function uses useState to track which FAQ is open.

```
1 import React, { useState } from 'react';
2 import './App.css'; // Importing our custom CSS
3
4 // The dataset mapping out the FAQs
5 const faqData = [
6   {
7     id: 1,
8     question: "What is the Virtual DOM?",
9     answer: "The Virtual DOM is a lightweight copy of the actual DOM. React uses it to figure out what parts of the
10   },
11   {
12     id: 2,
13     question: "Why do we need keys in React lists?",
14     answer: "Keys help React identify which items have changed, are added, or are removed. They should be given to
15   },
16   {
17     id: 3,
18     question: "What is the difference between state and props?",
19     answer: "Props are passed down from a parent component to a child component and are read-only. State is man
20   },
21   {
22     id: 4,
23     question: "Can multiple accordions be open at once here?",
24     answer: "No. Because we are storing a single integer (the ID of the active item) in our state instead of an
25   }
26 ];
27
28 function App() {
29   // Local state to track which FAQ is open. Default is null (all closed)
```

Screenshots:

